

计算机组成原理课程内容总结

1 课程基本信息

本课程为《计算机组成原理》，主要围绕计算机系统的组成、运行机制以及软硬件协同工作原理展开。课程强调从系统层面理解程序执行过程，培养学生的系统分析能力和系统设计能力。

2 参考书目

1. 袁春风等：《计算机组成与设计（基于 RISC-V 架构）》，高等教育出版社，2019 年。
2. 胡伟武等：《计算机体系结构基础（第二版）》，机械工业出版社，2018 年。
3. David A. Patterson 等著，王党辉等译：《计算机组成与设计：硬件/软件接口》，机械工业出版社，2015 年。
4. 袁春风等：《计算机系统基础》，机械工业出版社，2014 年。
5. 白中英：《计算机组成原理试题库及实现》，科学出版社，2010 年。
6. 白中英：《计算机组成原理》，电子工业出版社，2010 年。
7. Andrew S. Tanenbaum 等著，刘卫东等译：《计算机组成：结构化方法》，机械工业出版社，2014 年。
8. Yale N. Patt 等著，梁阿磊等译：《计算机系统概论》，机械工业出版社，2016 年。

3 本学期需要掌握的主要内容

本课程要求学生掌握计算机系统中的基本原理、机制、模型、标准以及实现方法。

3.1 原理与机制

需要重点理解以下内容：

- 程序的自动执行机制；
- CPU 的基本结构与工作过程；
- 虚拟存储器机制；
- Cache 缓存机制；
- 页面置换算法，如 LRU、LFU；
- 中断机制；
- DMA 机制；
- 程序执行过程中的异常检测与处理。

3.2 模型与标准

需要掌握计算机系统中常见的模型与标准，例如：

- 全加器与半加器；
- IEEE 754 浮点数标准；
- PCI 总线标准；
- USB 接口标准。

3.3 实现方法

课程还涉及一定的系统设计与硬件实现方法，包括：

- Vivado / ISE 工具的使用；
- FPGA 相关设计方法；
- 数字逻辑电路与计算机组成部件的实现。

4 考核方式

课程考核形式包括课程设计和考试。

总评成绩通常由以下部分组成：

- 作业；
- 课堂表现；
- 考勤；
- 期末考试；
- 课程设计。

5 第一讲：计算机系统组成概述

第一讲主要讨论为什么要学习《计算机组成原理》，并通过多个程序示例说明：程序运行结果不仅取决于高级语言代码本身，还与计算机底层结构、机器指令、数据表示、存储系统、操作系统以及硬件实现密切相关。

6 什么是计算机系统

计算机系统可以理解为硬件与软件的统一体。

- 硬件是计算机系统的躯体；
- 软件是计算机系统的灵魂。

只有从整体系统角度理解硬件和软件之间的关系，才能真正理解程序的执行过程。

7 为什么学习计算机组成原理

7.1 程序执行并不只由高级语言决定

程序员编写的高级语言程序最终必须被转换为机器指令，才能在计算机上执行。机器指令由二进制序列组成，是计算机硬件能够直接理解并执行的形式。

因此，程序的执行结果不仅取决于高级语言语法和语义，还取决于：

- 数据在机器中的表示方式；
- 机器指令的含义和执行过程；

- CPU 内部运算电路;
- 存储系统结构;
- 操作系统机制;
- 编译器和语言处理系统;
- 指令集体系结构 ISA;
- 微体系结构。

8 典型程序问题分析

8.1 数组求和程序中的异常

对于如下函数:

```
sum(int a[], unsigned len)
{
    int i, sum = 0;
    for (i = 0; i <= len - 1; i++)
        sum += a[i];
    return sum;
}
```

当参数 `len = 0` 时, 理论上返回值应该是 0, 但实际执行时可能发生访存异常。原因在于:

- `len` 是无符号整数;
- 当 `len - 1` 时发生无符号整数下溢;
- 循环条件可能被错误满足;
- 程序访问了非法内存地址;
- 异常与机器指令、虚拟地址空间和异常处理机制有关。

该问题说明, 理解程序执行结果需要掌握高级语言运算规则、机器指令执行、内部运算电路以及存储系统机制。

8.2 整数乘法溢出问题

若 x 和 y 均为 `int` 类型，当：

$$x = 65535$$

执行：

$$y = x \times x$$

可能得到：

$$y = -131071$$

这是因为计算机中的整数位数有限，乘法结果超过可表示范围时会发生溢出。现实世界中平方数总是非负的，但在计算机中，由于补码表示和有限字长，平方结果可能表现为负数。

该问题说明，计算机是一个有限字长的模运算系统。

8.3 整数取负与比较问题

对于任意 `int` 类型变量 x 和 y ，表达式：

$$(x > y) == (-x < -y)$$

并不总是成立。

当：

$$x = -2147483648$$

即最小的 32 位有符号整数时，对其取负会发生溢出，因为正数范围无法表示：

$$2147483648$$

因此，在计算机中一些数学上看似显然成立的性质，在有限字长和补码表示下并不一定成立。

8.4 动态内存分配中的整数溢出

对于如下代码：

```
int copy_array(int *array, int count) {  
    int i;
```

```
int *myarray = (int *) malloc(count * sizeof(int));
if (myarray == NULL)
    return -1;
for (i = 0; i < count; i++)
    myarray[i] = array[i];
return count;
}
```

当 `count` 很大时, 例如:

$$count = 2^{30} + 1$$

则:

$$count \times \text{sizeof}(int) = 4G + 4$$

该结果可能发生整数溢出, 导致实际申请的堆空间远小于需要的空间。之后循环复制大量数据时, 会破坏堆区数据。

该问题涉及:

- 乘法运算及溢出;
- 堆空间分配;
- 虚拟地址空间;
- 存储空间映射。

8.5 格式化输出与参数传递问题

对于代码:

```
#include <stdio.h>

int main()
{
    double a = 10;
    printf("a = %d\n", a);
    return 0;
}
```

程序使用 `%d` 输出 `double` 类型变量, 格式说明符与实际参数类型不匹配。在 IA-32 和 x86-64 平台上, 输出结果可能不同, 这是因为:

- `double` 采用 IEEE 754 标准表示；
- 不同体系结构的过程调用约定不同；
- IA-32 和 x86-64 的参数传递方式不同；
- 浮点数寄存器和整数寄存器的使用方式不同。

该例说明，程序行为不仅取决于 C 语言代码，也取决于体系结构和调用约定。

8.6 数组越界与栈布局问题

对于函数：

```
double fun(int i)
{
    volatile double d[1] = {3.14};
    volatile long int a[2];
    a[i] = 1073741824;
    return d[0];
}
```

当 `i` 取不同值时，可能会修改栈中的不同数据，从而导致返回值异常，甚至出现存储保护错误。

该问题涉及：

- 数据在机器中的表示；
- 过程调用机制；
- 栈中局部变量的布局；
- 数组越界访问；
- 存储保护机制。

8.7 数组访问顺序与 Cache 性能问题

两个程序功能相同，算法复杂度也相同：

```
void copyji(int src[2048][2048], int dst[2048][2048])
{
    int i, j;
```

```
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}

void copyij(int src[2048][2048], int dst[2048][2048])
{
    int i, j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

但是它们的执行时间可能差别很大。在某些处理器上，不同访问顺序可能导致约 21 倍的性能差异。

原因在于：

- C 语言二维数组按行优先存储；
- 按行访问具有较好的空间局部性；
- 按列访问容易导致 Cache 不命中；
- Cache 机制显著影响程序性能。

该问题说明，时间复杂度相同的程序，实际运行时间不一定相同。

9 系统思维

学习计算机组成原理的核心目标之一，是建立系统思维。

所谓系统思维，就是从计算机系统整体角度出发分析和解决问题。

9.1 系统思维的基本认识

- 高级语言语句必须转换为机器指令才能执行；
- 机器指令是由 0 和 1 组成的二进制序列；
- 计算机系统是有限字长的模运算系统；
- 运算器本身并不知道操作数是有符号数还是无符号数；
- 在计算机世界中，数学规律可能因溢出而失效；

- 内存访问比寄存器访问慢得多；
- 磁盘访问比内存访问慢得多；
- 进程具有独立的逻辑控制流和地址空间；
- 过程调用通常使用栈保存参数、返回地址和局部变量；
- 递归调用会带来额外指令开销，并可能导致栈溢出。

9.2 系统思维的重要性

只有理解计算机系统的整体结构，才能：

- 正确解释程序执行结果；
- 写出更安全、更高效的程序；
- 理解软硬件之间的关系；
- 分析系统性能瓶颈；
- 设计和改进计算机系统。

10 计算机系统的抽象层次

计算机系统由多个抽象层次组成，不同课程关注的层次不同，但这些层次之间相互联系。

程序执行结果不仅取决于算法和程序本身，还取决于：

- 高级语言；
- 编译器；
- 操作系统；
- 指令集体系结构；
- 微体系结构；
- 数字逻辑电路；
- 物理硬件。

学习计算机组成原理，就是要将这些层次联系起来，从整体上理解计算机系统的运行过程。

11 算术逻辑单元 ALU

11.1 ALU 的定义

算术逻辑单元，即 Arithmetic and Logical Unit，简称 ALU，是中央处理器 CPU 的执行单元，也是 CPU 的核心组成部分。

ALU 的主要功能是执行二进制的算术运算和逻辑运算。

11.2 ALU 的主要功能

ALU 通常能够完成以下操作：

- 加法运算；
- 减法运算；
- 与、或、非、异或等逻辑运算；
- 比较运算；
- 移位运算；
- 条件判断相关运算。

11.3 数据表示形式

在现代 CPU 体系结构中，整数数据通常采用二进制补码形式表示。补码表示使得加法和减法可以统一由加法器完成，从而简化硬件设计。

11.4 ALU 的历史

1945 年，数学家冯·诺依曼在 EDVAC 技术报告中首次提出了 ALU 的概念。

11.5 简单 1 位 ALU

简单 1 位 ALU 可以完成基本的 1 位算术与逻辑运算。多个 1 位 ALU 可以级联组成多位 ALU，例如 32 位 ALU 或 64 位 ALU。

简单 ALU 通常主要处理定点数，也就是整数运算。对于乘法和除法，简单 ALU 往往没有独立的乘除法电路模块，而是通过多次使用加法、减法和移位操作实现。

12 课程学习的核心结论

通过本讲内容可以得到以下结论：

1. 程序执行结果不仅由高级语言代码决定，还受到机器级表示、指令执行、存储结构、操作系统和硬件机制的影响。
2. 计算机系统是有限字长系统，整数溢出、浮点误差、地址越界等问题都可能影响程序行为。
3. 理解程序运行结果必须从系统层面出发，而不能只停留在高级语言层面。
4. Cache、虚拟存储器、过程调用、异常处理等机制都会影响程序的正确性和性能。
5. 学习计算机组成原理有助于建立系统思维，为后续学习操作系统、编译原理、计算机体系结构等课程打下基础。
6. 只有先理解系统，才能改革系统，并更好地应用和设计系统。